

Reverse Engineering Suicide Risk Prediction Models with MLOps and Explainable AI

Ms.M.RAJALAKSHMI,[AP/SS],
AI&DS

Dr. Mahalingam College of Engineering
and Technology Tamil Nadu, India

Maha Mathesh Raja B
Dept. of Artificial Intelligence
and Data Science
Dr. Mahalingam College of
Engineering and Technology
Tamil Nadu, India

Irfan Basha R
Dept. of Artificial Intelligence
and Data Science
Dr. Mahalingam College of
Engineering and Technology
Tamil Nadu, India

Sridhar M
Dept. of Artificial Intelligence and
Data Science
Dr. Mahalingam College of
Engineering and Technology
Tamil Nadu, India

Abstract – A trained language model can flag text that looks like it might signal suicidal ideation, which makes it a potentially useful decision-support signal. . But transformer classifiers are difficult to audit, difficult to explain to a nontechnical reader and difficult to keep running responsibly after the research phase. In this paper, the authors follow a reverse engineering approach to the BERT and MentalBERT-based suicide-risk classification, which integrates model auditing, explainable AI (XAI), MLOps, privacy-aware deployment, and model optimization. Instead of training a model and then calling it a day, we consider the model to be an inherited artifact and must understand it before using it responsibly. This text gets governed, anonymized, tokenized, passed through a fine-tuned transformer classifier, then each prediction is analyzed through various methods: SHAP token attribution, attention visualization, probing experiments, bias analysis, and adversarial robustness tests. On the bottom sits an engineering layer which versions data and models, keeps an eye on experiments, automates the validation course of, and compresses the model for edge inference. We tend to an on-device-first approach, and raw text remains local by default and only leaves the device with explicit consent. The project is in a stage where they report a 80/10/10 data split, consumer-device targets (inference), INT8 compression, and 78% conceptual alignment observation for high-risk predictions. These are project stage observations only and not a statement of clinical effectiveness since the implementation is being built out. Our goal is not to have an autonomous clinical decision maker, but transparency, auditability, privacy and human oversight throughout.

Suicide risk prediction, reverse engineering, MentalBERT, BERT, explainability of AI, SHAP, MLOps, model compression, on-device inference, privacy, responsible AI

I. INTRODUCTION

Traditional mental-health risk assessment has been dependent on face-to-face interaction with a trained mental-health practitioner: structured questionnaires, years of clinical experience, and judgement. What NLP brings to that picture is not a new one, but another one, as people frequently experience feelings of distress, hopelessness, isolation, or suicidal thoughts before they put them into words with others. Previous research has demonstrated that social media language provides valuable information for mental-health and suicide-risk assessment, and that multimodal or knowledge-aware methods can be used to further narrow this contextual picture [1]–[4]. Here transformer architectures that concurrently attend to both left and right context are useful, which can also be fine-tuned to a downstream classification task [5]. Domain-specific

Further, the pre-training takes it further: MentalBERT has been able to enhance performance in mental-health detection tasks in particular [6]

Despite all these developments, there is still a deployment gap that needs to be addressed. The class of a classifier does not always tell a developer or clinician what evidence in the text makes that determination, and that is where an auditability problem comes in: a developer or clinician might be confident that the model is making a prediction of risk, but not confident that it is picking up on something clinically relevant, or on a spurious correlation, or a demographic artifact, or a quirk of the training set. The most significant concerns with cloud-centric inference are that it can reveal sensitive text, while most of the research prototypes were not designed for versioning, monitoring, regression testing, or deployment controls.

A. Problem Statement:

Existing suicide-risk classifiers can produce useful predictions, but they commonly come with black-box behavior, thin production infrastructure, privacy and latency concerns, and little thought given to cross-platform optimization. The problem this work sets out to address is how to systematically reverse engineer a trained transformer classifier and turn what is learned from that process into an auditable, maintainable, privacy-aware decision-support pipeline — rather than just a model behind an API.

B. Objectives

- Identify the linguistic, emotional, contextual, and representation-level patterns that are actually associated with each prediction.
- Build an MLOps lifecycle that covers data governance, experiment tracking, model versioning, testing, deployment, and monitoring.
- Evaluate INT8 quantization, pruning, and knowledge distillation as routes to resource-constrained inference.
- Provide clinician-oriented explanations through SHAP, attention visualization, and clinical-concept mapping.
- Audit subgroup behavior and robustness before any of this is considered for real-world clinical use.

II. EXISTING SYSTEM

What exists today spans keyword and rule-based methods, classical machine learning, recurrent neural networks, convolutional architectures, and transformer-based classifiers. The earlier systems in that list are comparatively simple and tend to miss long-range context; transformers do a better job of

capturing contextual dependencies, but that comes at the cost of an internal behavior that is genuinely hard to inspect. Some work has gone further and explored severity classification using clinical knowledge and C-SSRS-inspired labels, and multimodal research has shown that text-based signals can be strengthened by adding behavioral, relational, and visual signals on top [2]–[4].

TABLE I. LIMITATIONS OF EXISTING SUICIDE-RISK PREDICTION SYSTEMS

Limitation	Effect	Why It Matters
Black-box prediction	Only a class or score is returned	Clinicians cannot easily audit evidence
Cloud-first inference	Sensitive text may leave device	Privacy and governance concerns
Research-prototype lifecycle	Manual model/data handling	Weak reproducibility and monitoring
Limited robustness auditing	Spurious patterns may remain	Brittle predictions
Limited edge optimization	Large models require more compute	Restricts mobile/edge deployment
Dataset bias	Distributional artifacts may be learned	Unequal subgroup performance

Our literature review kept turning up the same handful of representative directions — EEG-based depression detection, XGBoost paired with SHAP, MentalBERT, and dual-context BERT — and none of them combine XAI, reverse engineering, MLOps, privacy, and deployment optimization in one place. That combination is the gap this work is aimed at.

A. Related-Work Comparison Matrix

Table II lines the proposed system up against the representative prior approaches surfaced in our literature review.

TABLE II. COMPARISON WITH REPRESENTATIVE PRIOR APPROACHES

Study	Core Method	Strength	Limitation / Gap	Proposed Positio
Penava et al., 2024 [9]	1D CNN on resting-state EEG	Low compute; fast inference	EEG hardware; depression-focused; limited XAI/deployment	Text NLP + XAI + MLOps + edge deployment
Ji et al., 2022 [6]	MentalBERT	Domain-specific representation	Black-box; limited deployment focus	Adds reverse engineering, XAI, MLOps, privacy
Matero et al., 2019 [10]	Dual-context BERT	Uses post + user history	Interpretability/deployment gap	Adds modern XAI and production lifecycle
Tang et al., project matrix	XGBoost + SHAP	Interpretable feature importance	Traditional ML for complex text	Transformer representations + multiple XAI methods

Looking across the row, no single prior direction pulls together modern transformer representations, multi-method explainability, a full MLOps lifecycle, privacy-aware deployment, and edge optimization all at once — which is exactly the combination the proposed system targets.

III. PROPOSED SYSTEM

Rather than a single neural network, the proposed system is really a stack of layers. The first governs sensitive data through consent, minimization, anonymization, and secure storage. The second handles preprocessing and runs transformer inference. The third is where the model gets

reverse engineered, using both local and global explanations. The fourth operationalizes everything through MLOps, and the fifth compresses the model and pushes it out to supported devices. Monitoring closes the loop at the end, feeding what is observed in production back into risk prediction.

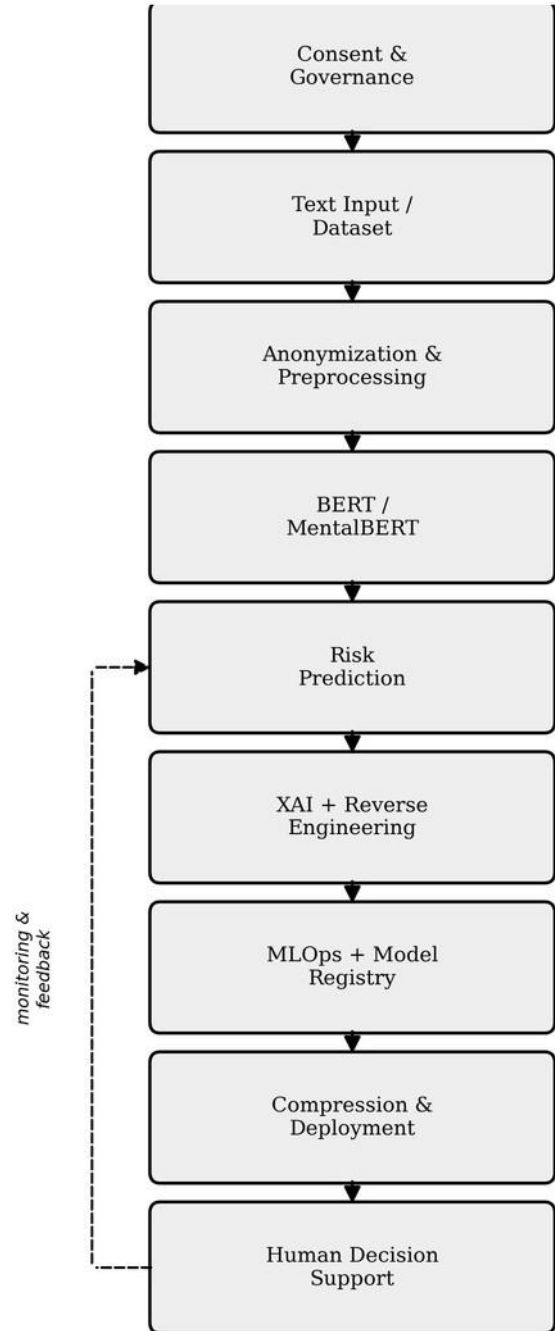


Fig. 1. Proposed reverse-engineering, explainability, MLOps, and deployment flow.

A. Risk Classification Formulation

Four conceptual classes are used: No Risk, Low Risk, Moderate Risk, and High Risk, with the mapping informed by C-SSRS-style severity assessment. For input text x , the transformer produces class probabilities as follows:

$$p(y = k | x) = \text{softmax}(z\theta(x))_k, k \in \{1, \dots, K\} \quad (1)$$

The predicted class is simply whichever category comes out with the highest probability. It is worth restating plainly that this output is a research decision-support signal, not a diagnosis.

B. Proposed Contributions

- Reverse engineering an inherited, already fine-tuned transformer instead of treating it as an opaque endpoint.
- Combining token-level SHAP attribution, attention inspection, and representation probing rather than relying on any one of them alone.
- Integrated bias and adversarial robustness auditing.
- An end-to-end MLOps setup built on DVC, MLflow/Weights & Biases, Optuna, GitHub Actions, and monitoring.
- Model compression and export for ONNX/TFLite/Core ML-oriented deployment.
- An on-device-first privacy strategy with an explicit-consent cloud fallback.

IV. METHODOLOGY

A. Data Governance and Preprocessing

The primary dataset direction here is the UMD Reddit Suicidality Dataset, with CLPsych-style resources and clinical-note data on the table for broader validation later. Mental-health text can carry sensitive personal information, so governance has to start before the model ever sees the data: identifiers get removed or pseudonymized, access is role-controlled, and retention is kept to a minimum.

Preprocessing tokenizer-compatible segmentation, truncation/padding, and preserving whatever context is clinically meaningful — negation and other contextual cues should not just get stripped out along the way.

B. Transformer Classification

BERT builds bidirectional contextual representations and can be fine-tuned with a classification head on top [5]. MentalBERT takes that a step further through domain-specific pretraining on mental-health-related text [6]. We compare four architectures here: Bi-LSTM+GloVe, BERT-base-uncased, MentalBERT, and RoBERTa-large.

TABLE III. MODEL ARCHITECTURES COMPARED

Model	Role	Expected Characteristic
Bi-LSTM + GloVe	Pre-transformer baseline	Lower capacity
BERT-base-uncased	General transformer baseline	Strong contextual representation
MentalBERT	Domain-adapted transformer	Mental-health language alignment
RoBERTa-large	High-capacity benchmark	Higher compute; upper benchmark range

C. Reverse Engineering with SHAP

SHAP assigns additive feature-importance values to individual predictions [7]. For text, tokens play the role of features, and their contributions can be aggregated across many examples at once. A simplified way to write this is:

$$f(x) \approx \phi_0 + \sum_i \phi_i \quad (2)$$

Tokens that push toward a target risk class get highlighted, while tokens pushing away from it show up as evidence against that class. When aggregated globally, recurring semantic clusters tend to emerge — hopelessness, isolation, urgency, or method-related language, for instance — though these clusters need to be read cautiously, since correlation here does not amount to clinical causation.

D. Attention Visualization

Transformer attention matrices can be inspected layer by layer and head by head to see which tokens are receiving the most attention. BertViz-style visualization is useful for studying head specialization and how attention patterns differ across risk classes. We treat attention as a diagnostic signal rather than a full causal explanation on its own, which is why it is combined with SHAP and the other analyses below.

E. Probing Experiments

A linear probe trained on frozen representations can test whether linguistic properties such as negation, tense, or affect are actually encoded in the hidden states:

$$\hat{y}_{\text{probe}} = \text{softmax}(Wh + b) \quad (3)$$

A high probe score tells you the information is represented somewhere in the network — it does not tell you the original classifier is actually using that information to make its decision.

F. Bias and Fairness Audit

Subgroup evaluation can look at demographic or contextual variables when doing so is both ethically and statistically justified. The metrics here are demographic parity, equal opportunity, and equalized odds; equal opportunity, for instance, compares true-positive rates across groups. Where mitigation is needed, reweighting or threshold calibration are the two options we consider.

$$\text{TPR} = \text{TP} / (\text{TP} + \text{FN}) \quad (4)$$

$$\text{FPR} = \text{FP} / (\text{FP} + \text{TN}) \quad (5)$$

G. Adversarial Robustness

This part of the framework checks whether small changes to the input text cause the prediction to swing wildly. Controlled character perturbations and word substitutions are a good way to surface that kind of brittleness — the most telling failures are the ones where semantically similar text produces a large probability shift without any real change in intended meaning.

H. MLOps Pipeline

The lifecycle runs Data Layer → Preprocessing → Training → Model Registry → Compression → Deployment → Monitoring. DVC versions the datasets, MLflow/W&B tracks experiments, Optuna tunes hyperparameters, GitHub Actions automates testing, and Evidently-style monitoring watches for drift. All told, that is a six-phase lifecycle running from data governance through to ongoing monitoring.

TABLE IV. MLOPS STAGES AND TOOLING

Stage	Technique/Tool	Purpose
Data versioning	DVC	Reproducible datasets
Experiment tracking	MLflow / W&B	Parameters, metrics, artifacts
Hyperparameter tuning	Optuna	Hyperparameter search
Training	Hugging Face Accelerate	Efficient training
CI/CD	GitHub Actions	Automated tests/regression
Monitoring	Evidently AI	Drift/performance monitoring
Containerization	Docker	Reproducible runtime

SHAP to estimate token contributions, attention visualizations display a layer/head pattern, and probing experiments check whether particular concepts show up in the hidden states..

L. Training Procedure

Training begins with a specific split of the dataset and a preprocessing configuration that can be reproduced exactly. The tokenizer and the model checkpoint are given version numbers, then supervised fine-tuning runs, with validation performance it is checked following each epoch, and then the best checkpoint is selected. using a safety-relevant metric — macro-F1 or high-risk recall — not just accuracy.

TABLE VI. TRAINING AND VALIDATION PROCEDURE

Step	Procedure	Validation Check
1	Load governed dataset	Missing values, duplicates
2	Create 80/10/10 split	No leakage across partitions
3	Tokenize text	Sequence length statistics
4	Fine-tune transformer	Loss and macro-F1
5	Select checkpoint	Validation high-risk recall
6	Run XAI audit	Attribution stability
7	Run robustness/fairness audit	Failure-rate comparison
8	Compress/export	Accuracy and latency retention
9	Register and deploy	Model/version traceability

Model/version traceability

For a K-class problem, The basic optimization goal for K class problem is cross-entropy loss. With y as the one-hot target and p as the predicted probability for class i , the loss over N samples works out to:

$$LCE = -(1/N) \sum_n \sum_k y(n,k) \log p(n,k) \quad (10)$$

Class weighting should be done when risk categories are skewed, but it should be optimized with the validation data and documented in the experiment registry in such a way that any enhancement that may be obtained is actually repeatable later.

M. Reverse-Engineering Audit Protocol

TABLE VII. REVERSE-ENGINEERING AUDIT PROTOCOL

Audit	Question Answered	Evidence Produced
Token attribution	Which words influence a prediction?	SHAP heatmap/ranking
Attention inspection	Where does the transformer attend?	Head/layer visualization
Concept probing	Is a linguistic concept encoded?	Probe score
Counterfactual test	Does a small semantic change alter output?	Probability delta
Fairness test	Does performance differ across groups?	TPR/FPR/F1 gaps
Calibration	Do probabilities reflect observed frequency?	Calibration curve
Drift test	Has input distribution changed?	Drift score/report

IM. odel Compression and Deployment

Large transformers just do not run well on mobile and edge hardware as is, so this stage looks at INT8 quantization, pruning, and knowledge distillation as ways around that. A simplified symmetric quantization relationship looks like this:

$$q = \text{round}(w / s), \hat{w} = s \cdot q \quad (6)$$

At this stage, the project reports roughly a four-fold reduction in size with better than 97% accuracy retention for INT8 quantization. That number should be read as a project-stage observation for now, one that still needs revalidating on the final trained model rather than treated as an established clinical benchmark.

J. Detailed Architectural Layers:

The implementation is divided into layers that are interoperable so that That development, auditing, deployment and monitoring do not have to go hand-in-hand. The data-governance layer is responsible for collecting and retaining data, while the NLP layer converts text into The data-governance layer is responsible for collecting and retaining data, and the NLP layer turns text into The model-ready representations are made to the inference layer, which generates risk.Reverse engineering layer dives into probabilities; why Those predictions occur; and the MLOps layer takes care of the lifecycle end to end. A model may be replaced without having to redesign the entire application to

TABLE V. ARCHITECTURAL LAYERS OF THE IMPLEMENTATION

Layer	Main Components	Output
Governance	Consent, anonymization, access control, encryption	Governed input
NLP preprocessing	Cleaning, tokenization, padding/truncation	Token IDs + masks
Model	BERT / MentalBERT + classification head	Class probabilities
XAI	SHAP, attention, concept probing	Attributions + concepts
Audit	Bias, robustness, calibration	Risk/failure report
MLOps	DVC, MLflow/W&B, CI/CD, monitoring	Versioned model
Deployment	ONNX Runtime, TFLite, edge/mobile	Local inference

K. End-to-End Processing

Preprocessing: Take an input document x and return a token sequence $T = \{t_1, t_2, \dots, t_n\}$. The tokenizer converts each token into an integer identifier and constructs a mask for attention, and the transformer converts this into "contextual hidden states" $H = \{h_1, h_2, \dots, h_n\}$ (a term coined by Google). A classification representation is then fed into a fully connected layer and a softmax:

$$H = \text{Transformer}(\theta(T, M)) \quad (7)$$

$$z = W \cdot h_{CLS} + b \quad (8)$$

$$p = \text{softmax}(z) \quad (9)$$

The model's predicted category is the probability that results in the highest value. In the reverse engineering process, the same input is fed into the explanation modules:

N. Security and Data Handling

Sensitive text has no business sitting in ordinary application logs. Production logging should capture identifiers, timestamps, model versions, latency, and aggregate diagnostics — not raw user text, unless that has been explicitly authorized. Encryption belongs both in storage and in transit, and access should follow least-privilege principles throughout. A model registry should hold immutable versions of approved artifacts, and retired models need to be clearly marked so they cannot be deployed by accident.

O. Deployment Strategy

Here, the on-device inference is the best choice, primarily to reduce the amount of sensitive text being transmitted, as well as provide more predictable latency. The compressed model is exported to an interoperable runtime (e.g. ONNX Runtime or a suitable mobile runtime), and a cloud fallback is only provided if the user explicitly agrees and if the deployment environment has the appropriate

TABLE VIII. DEPLOYMENT OPTIONS

Deployment Option	Advantage	Trade-off
On-device	Privacy, low network dependency	Device compute/memory constraints
Private server	Centralized control and updates	Requires secure infrastructure
Cloud API	Easy scaling	Sensitive text leaves device
Hybrid	Balances availability and privacy	More complex governance

P. Evaluation Protocol

Evaluation runs on a held-out test set that never touches model selection. We report macro-F1, per-class precision/recall/F1, AUROC where it applies, confusion matrices, and confidence intervals. High-risk recall gets reported on its own too, given how much more it matters than the other classes, and calibration is checked so that a probability score does not get mistaken for a clinically meaningful certainty.

$$F1 = 2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall}) \quad (11)$$

On the deployment side, evaluation should also report median and tail latency, memory consumption, model size, throughput, and energy use wherever that is measurable. Compression only counts as successful if the drop in resource behavior easier to inspect, not to turn textual patterns into an requirements does not come with an unacceptable hit to safety-tomated diagnosis. relevant metrics.

TABLE IX. RECOMMENDED PERFORMANCE METRICS

Metric	Definition	Purpose
Accuracy	(TP+TN)/All	Overall correctness
Precision	TP/(TP+FP)	Controls false alarms
Recall	TP/(TP+FN)	Important for high-risk detection
F1	2PR/(P+R)	Balances precision and recall
Macro-F1	Mean class F1	Useful for imbalance
AUROC	Area under ROC	Ranking discrimination
Latency	Time/sample	Edge usability
Model size	Serialized MB	Deployment footprint

TABLE X. ERROR ANALYSIS FRAMEWORK

Error Type	Possible Cause	Recommended Investigation
False positive	Spurious distress-related vocabulary	SHAP + counterfactual review
False negative	Implicit or indirect expression	Context analysis + missed-concept audit
Class confusion	Adjacent severity categories	Confusion matrix + calibration
Subgroup error	Distributional imbalance	Stratified metrics
Robustness failure	Sensitivity to minor edits	Perturbation testing
Deployment regression	Compression/runtime change	Pre/post export benchmark

Error Type	Possible Cause	Recommended Investigation
Robustness failure	Sensitivity to minor edits	Perturbation testing
Deployment regression	Compression/runtime change	Pre/post export benchmark

R. Reproducibility and Model Governance

Every released model needs a unique version identifier tied to its training data version, preprocessing configuration, hyperparameters, source-code commit, evaluation report, and deployment artifact. A model promotion workflow can move through Development → Validation → Approved → Production states, and a rollback to a prior production version should always be an option if monitoring flags unexpected degradation.

TABLE XI. REPRODUCIBILITY AND GOVERNANCE ARTIFACTS

Artifact	Versioned With	Why
Dataset	DVC hash/version	Prevents silent data changes
Code	Git commit	Reproduces implementation
Model	Registry version	Controls deployment
Configuration	YAML/JSON + run ID	Reproduces experiments
Evaluation report	Model version	Supports audit
Runtime	Container/export version	Detects deployment changes

Explainability gets a further boost by grouping token-level evidence into higher-level clinical concepts — mapping several semantically related expressions onto broader ideas like hopelessness, social isolation, perceived burden, urgency, or help-seeking. That mapping should be reviewed by qualified domain experts rather than generated purely from the model's own explanations. The point is to make the model's behavior easier to inspect, not to turn textual patterns into an requirements does not come with an unacceptable hit to safety-tomated diagnosis.

S. Ethics, Privacy and Responsible Deployment

Given that this sits in a high-stakes mental-health domain, good engineering on its own is not enough. The framework leans on four familiar principles — beneficence, non-maleficence, autonomy, and justice. Consent needs to be meaningful, data collection kept to a minimum, and users told plainly that automated predictions are not diagnoses. Human reviewers keep the final say. On the protective-mechanism side, we're looking at differential privacy, encryption, anonymization/pseudonymization, access controls, audit logs, and model watermarking, alongside on-device inference by default, an optional cloud fallback with explicit consent, and readiness for federated learning down the line. These are engineering safeguards, and it is worth being clear that they do not by themselves amount to legal or clinical certification.

For deployment in India specifically, the final system would still need review against applicable data-protection, institutional requirements, and the privacy law is described here are presented as engineering measures, not as a claim of legal certification or clinical approval.

V. RESULTS AND DISCUSSION

A. Experimental Setup

The setup here is an 80/10/10 split for training, validation, and testing, A100-class hardware for training, and consumer

devices as the inference target. The final implementation is still under active development, so what follows separates reported project-stage observations from measurements that still need to be reproduced end to end on the completed system.

TABLE XII. EXPERIMENTAL SETUP

Parameter	Project-Stage Setting
Dataset split	80% train / 10% validation / 10% test
Training hardware	NVIDIA A100-class GPU
Inference	Consumer / mobile / desktop targets
Models	Bi-LSTM, BERT-base, MentalBERT, RoBERTa-large
XAI	SHAP, attention visualization, TCAV/concept mapping
Compression	INT8 quantization, pruning, distillation
Deployment	ONNX Runtime, TFLite, Core-ML-oriented workflow
Monitoring	Drift, latency, regression, feedback

B. Reported Project-Stage Findings

Summing up the results so far: MentalBERT is better than BERT, quantization loses just a fraction of the original accuracy, the explanations generated by SHAP seem clinically sensible on a case-by-case basis, and bias gaps have narrowed, and inference on the device is still possible. It should also be emphasized that the implementation has been on-going and that the deployment strategy has been modified several times during the process, thereby these numbers should be interpreted as a snapshot of the process and are not a final clinical validation study.

TABLE XIII. CONSOLIDATED PROJECT-STAGE RESULTS

Evaluation Area	Reported Status / Observation	Interpretation
Model comparison	MentalBERT reported to outperform base BERT	Domain-adapted pretraining is promising
Model compression	$\approx 4\times$ size reduction target/observation with INT8	Supports edge deployment; revalidate accuracy
Clinical-concept alignment	78% alignment reported for high-risk predictions	Promising XAI signal; needs larger expert study
Bias mitigation	Bias gaps reported as reduced	Needs subgroup sample sizes and confidence intervals
On-device inference	Feasibility reported	Needs device-specific latency/energy benchmarks
MLOps	DVC + experiment tracking + CI/CD + monitoring planned	Improves reproducibility and lifecycle control

C. Planned Final-Benchmark Comparison

Once the final trained and compressed models have been reproduced end to end, the metrics will be added to Table XIV, with many of the cells intentionally left blank to be reproduced right now, to ensure that design targets stay well away from the actual measurements

TABLE XIV. PLANNED FINAL-BENCHMARK COMPARISON

Metric	BERT	MentalBERT	Compressed Model
Accuracy	To be reproduced	To be reproduced	Target: >97% retention
Macro-F1	To be reproduced	To be reproduced	To be reproduced
High-risk Recall	To be reproduced	To be reproduced	To be reproduced
AUROC	To be reproduced	To be reproduced	To be reproduced
Model Size	Baseline	Baseline	$\approx 4\times$ reduction target
Latency	Device benchmark required	Device benchmark required	Expected lower

D. Discussion

The real value in this framework was never going to be a single accuracy number. What it does instead is connect a prediction to model behavior, and then connect that behavior to deployment controls. SHAP surfaces influential tokens, attention visualization exposes contextual interactions, probing tests whether linguistic information is actually encoded, fairness analysis flags subgroup differences, and robustness testing exposes instability — put together, that is a model audit, not just an explanation.

The MLOps layer is what gives all of this traceability. Mental-health classifiers can degrade quietly when language distributions shift, when data collection practices change, or when a new model version goes out the door. Versioning, automated regression checks, model registries, and drift monitoring all help keep that reproducible. On-device inference cuts down exposure of raw text, but it does not eliminate risk entirely — model extraction, compromised devices, careless logging, and users simply misreading the output are all still real concerns.

The INT8 result matters most on the deployment side, since model size feeds directly into memory and transfer requirements — though compression really needs to be judged against high-risk recall and macro-F1, not size alone. The 78% concept-alignment figure suggests explanations line up with human-interpretable clinical concepts in a good chunk of high-risk predictions, but that number still needs blinded expert annotation, inter-rater agreement, and confidence intervals behind it before anyone treats it as evidence of clinical validity. If there is one thing worth taking away from all of this, it is that the strongest part of the proposed system is the way prediction, explanation, governance, and deployment are tied together — this is better described as an auditable AI decision-support pipeline than as an autonomous suicide-risk detector.

E. Result Summary

TABLE XV. RESULT SUMMARY

Finding	Status	Implication
Domain adaptation	Promising	Continue MentalBERT evaluation
XAI	Promising	Validate explanations with experts
Compression	Promising	Benchmark safety metrics after quantization
Privacy	Design implemented/planned	Prefer local inference and minimize logs
MLOps	Architecture defined	Automate reproducibility and monitoring
Clinical readiness	Not established	Prospective validation is required

F. Limitations

- Dataset diversity is limited, especially where social-media data dominate training.
- Text-only modeling cannot capture the full clinical picture.
- The explainability methods used here are approximations, not proof of causation.
- Clinical validation and expert evaluation are still limited at this stage.
- English-centric training may hurt performance on multilingual populations.
- False positives and false negatives remain possible, and the model should never be the sole basis for a clinical decision.
- The deployment and performance values reported here are project-stage observations that still need final, reproducible benchmarking.
- Dataset shift is a real possibility — social-media language can differ substantially from clinical communication.
- There is label uncertainty: risk categories inferred from text may not match an expert clinical assessment.
- Sampling bias is possible, since users represented in the dataset may not represent the wider population.
- Explanation instability is possible — different XAI methods can emphasize different evidence for the

same prediction.

- Hardware variability affects latency and energy measurements, which depend heavily on device and runtime.
- Human factors are a genuine risk: users may over-trust or misinterpret an automated risk score.

G. Research Significance

What this work contributes is really an engineering perspective on mental-health NLP — treating the model lifecycle itself as something that can be audited, rather than treating the model as a finished product. Bringing together reverse engineering, XAI, MLOps, compression, and privacy builds a bridge between a research model and something that could actually be deployed. That matters more here than in most domains, because when a model works on sensitive text, predictive performance on its own — without traceability, governance, and robustness behind it — is not enough to justify responsible adoption.

VI. CONCLUSION

This paper presented a reverse-engineering framework for BERT and MentalBERT based Suicide-risk prediction covering aspects of explainability, MLOps, robustness auditing, model compaction and privacy-driven deployment. This is not a classifier-only approach because a classifier only answers the question of whether a model predicts risk; this one gets further and asks what the

Whether its behaviour is stable and fair, how it reacts to compression, how the entire lifecycle can be controlled after it is released into the world, model has actually learned

The evidence suggests most of the above—domain-adapted transformers, XAI, quantization, and on-device inference—are all part of a viable auditable mental health AI pipeline at this stage. This is still a research and engineering model however and is not a clinically tested diagnostic system. Future research should aim to scale up and diversify data sets, model in multiple languages, model with multiple modalities, systematically evaluate models by clinicians, enable federated learning and provide enhanced privacy assurances, and conduct prospective model validation. The ultimate goal throughout is clear, privacy-preserving decision support — outputs from the models can always be checked, challenged, followed, and overruled by a human expert.

ACKNOWLEDGMENT

The authors would like to thank Ms. M. Rajalakshmi, Assistant Professor (Senior Scale) in the Department of Artificial Intelligence and Data Science, Mr. Mahalingam college of engineering and technology for taking care of this project and providing us the computational resources and research environment.

REFERENCES

- [1] G. Coppersmith, M. Dredze, C. Harman, K. Hollingshead, and M. Mitchell, "CLPsych 2015 Shared Task: Depression and PTSD on Twitter," in *Proc. 2nd Workshop Comput. Linguistics Clin. Psychol.*, 2015.
- [2] D. Ramírez-Cifuentes, A. Freire, R. Baeza-Yates, J. Puntí, P. Medina-Bravo, D. A. Velazquez, J. M. Gonfaus, and J. González, "Detection of Suicidal Ideation on Social Media: Multimodal, Relational, and Behavioral Analysis," *J. Med. Internet Res.*, vol. 22, no. 7, e17758, 2020.
- [3] M. Gaur, A. Alambo, J. P. Sain, U. Kursuncu, K. Thirunarayan, R. Kavuluru, A. Sheth, R. S. Welton, and J. Pathak, "Knowledge-Aware Assessment of Severity of Suicide Risk for Early Intervention," in *Proc. WWW*, 2019, pp. 514–525.
- [4] M. Tadesse, H. Lin, B. Xu, and L. Yang, "Detection of Suicide Ideation in Social Media Forums Using Deep Learning," *Algorithms*, vol. 13, no. 1, p. 7, 2020.
- [5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [6] S. Ji, T. Zhang, L. Ansari, J. Fu, P. Tiwari, and E. Cambria, "MentalBERT: Publicly Available Pretrained Language Models for Mental Healthcare," in *Proc. LREC*, 2022.
- [7] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems* 30, 2017.
- [8] M. Ribeiro, S. Singh, and C. Guestrin, "Why Should I Trust You?": Explaining the Predictions of Any Classifier," in *Proc. KDD*, 2016, pp. 1135–1144.
- [9] P. Penava and R. Buettner, "Early-Stage Non-Severe Depression Detection Using a Novel Convolutional Neural Network Approach Based on Resting-State EEG Data," *IEEE Access*, vol. 12, pp. 173380–173389, 2024.
- [10] M. Matero *et al.*, "Suicide Risk Assessment Using Machine Learning and Natural Language Processing," *arXiv preprint*, 2019.